

Token Compression and Routing, how to control agentic coding costs

First, you make it work → Then you make it efficient

Sacha Morard

co-founder · edgee.ai



How much does a developer spend on tokens per month?

How much does a developer spend on tokens per month?

\$150-250

Anthropic official numbers

Source: <https://code.claude.com/docs/en/costs>

How much does a developer spend on tokens per month, through Edgee gateway?

How much does a developer spend on tokens per month, through Edgee gateway?

\$1,039

per engineer, per month measured by Edgee

[Gartner Peer Insights](#), June 2026

23% at \$200-500/dev/month

6% above \$2,000

// what nobody budgeted for

18 months ago this line did not exist in any engineering budget.

Today it is the line the CFO asks about first.

Uber

Annual AI coding budget exhausted in 4 months. Now capped at \$1,500 per employee per month.

Microsoft

Internal Claude Code licences cancelled, effective 30 June 2026. Engineers moved to Copilot CLI.

Meta

Internal memo to 6,000 staff: internal AI usage on track to cost billions in 2026, excluding capex.

Deloitte

Published a CFO guide to token economics, April 2026. The topic did not exist 18 months earlier.

Sources:

<https://techcrunch.com/2026/06/02/uber-caps-employee-ai-spending-after-blowing-through-budget-in-four-months/>

<https://www.techradar.com/pro/microsoft-may-discontinue-claude-code-internally-as-it-looks-to-push-users-towards-github-copilot>

The budget broke at the agent layer, not the app layer

Two years ago AI meant features inside applications, and a gateway meant one endpoint your own code called. That surface still exists. It is no longer where the budget breaks.

\$4B

enterprise spend on agentic coding
in 2025, up ~5x in a year

55%

of departmental AI budgets now
go to coding agents (Menlo
Ventures)

2x / 6 mo

our customers' bills double every
six months, with no cost per team

So what actually has to be solved.

Five problems, and "use fewer agents" is not a valid solution.

// WHAT THE EXPLOSION BROKE

- No visibility
- No enforceable budget
- No resilience
- No model flexibility
- No value attribution

So what actually has to be solved.

Five problems, and "use fewer agents" is not a valid solution.

// WHAT THE EXPLOSION BROKE

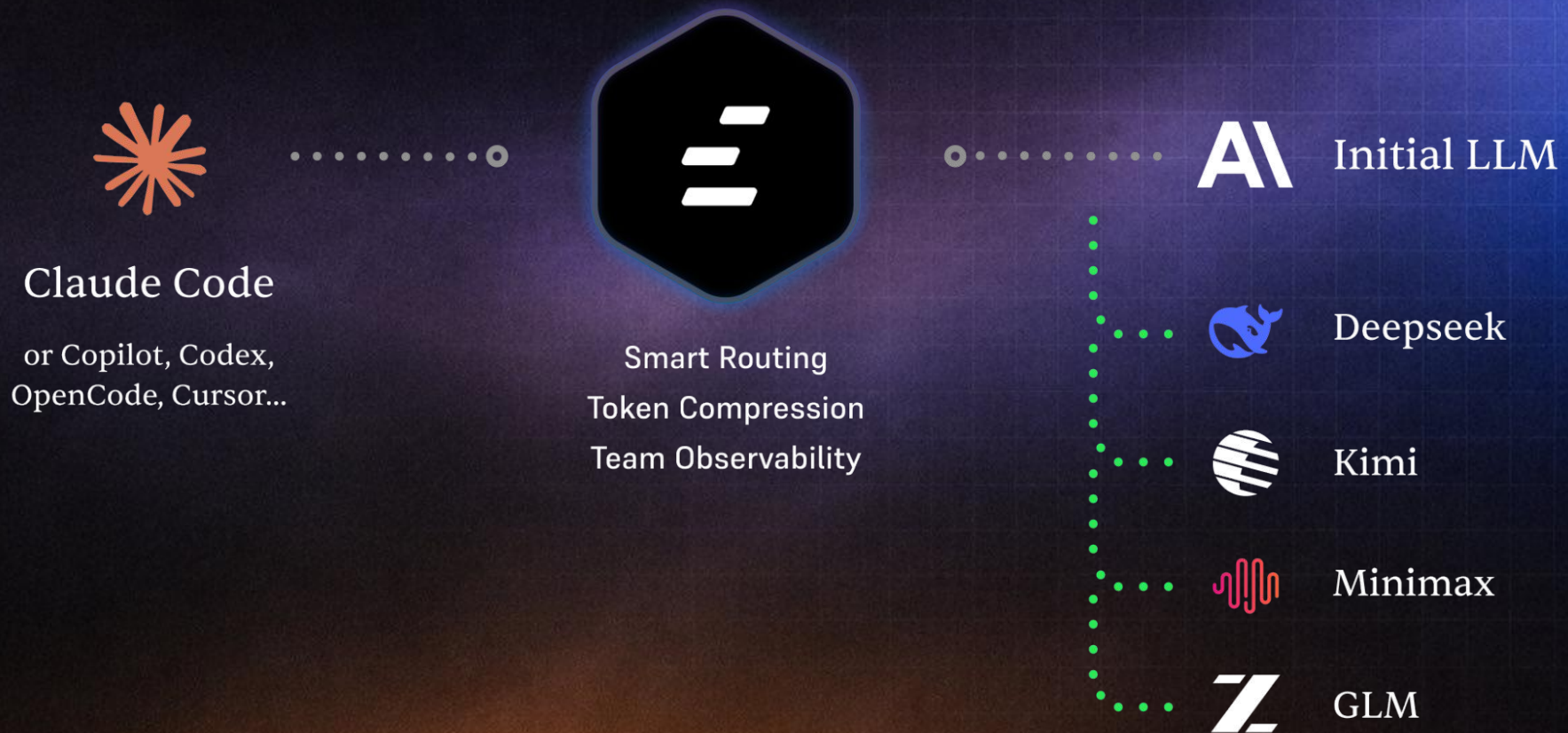
- No visibility
- No enforceable budget
- No resilience
- No model flexibility
- No value attribution

// WHAT THIS TALK COVERS

1. Observability
2. Token compression
3. Routing



Agent Gateway, made for coding assistants





Sacha Morard

Edgee: edgee.ai/u/sacha

- Self-taught code lover
- Technological strategist
- Entrepreneur

Musician	Designer	Developer	Tech Entrepreneur	
2001	2008	2010	2013	Date.now()

Observability

Tokens are the **new** bandwidth.

Same trajectory. Same solution pattern. The web built routing, compression and observability. The AI era needs all three, in that order.

// WEB ERA

Observability

measure everything

Gzip / Brotli

compress payloads

CDN / Edge

route intelligently

// AI ERA

Token metering

observe everything

Token compression

compress context

Agent Gateway

route + fallback

What the community **built**.

// LOCAL USAGE · OPEN SOURCE

ccusage

Turns local agent logs into token and estimated cost reports.

Tokscale

Combines token usage and estimated costs across coding agents.

// EXECUTION TRACES · OPEN SOURCE

Langfuse

Open-source tracing with token, cost and latency tracking.

OpenLIT

OpenTelemetry tracing for agent steps, tokens and costs.

// PROVIDER DASHBOARDS · PROPRIETARY

Claude Console API usage and costs by model and workspace. CSV exports.

OpenAI API usage and spend by project and model. CSV exports.

Follow the cost. Find the cause.

DASHBOARD

→ TEAM

→ SESSION

→ REQUEST

Where does it go?

Spend & savings
Model / provider mix
Cache hit rate

Who uses it?

Member / squad
Coding agent
Usage over time

For which work?

Repository / PR
Cost · calls · duration
Models used

What drives it?

Input / cache / output
Route · billing · latency
Compression savings

Then connect usage to outcomes.

Accepted changes · tests passed · rework

Token Compression

Two places where **tokens** can be saved.

LAYER 01

Input

// what enters the context
99% of the token volume

- System prompts
- Tool results
- Codebase context
- Conversation history
- MCP tool definitions

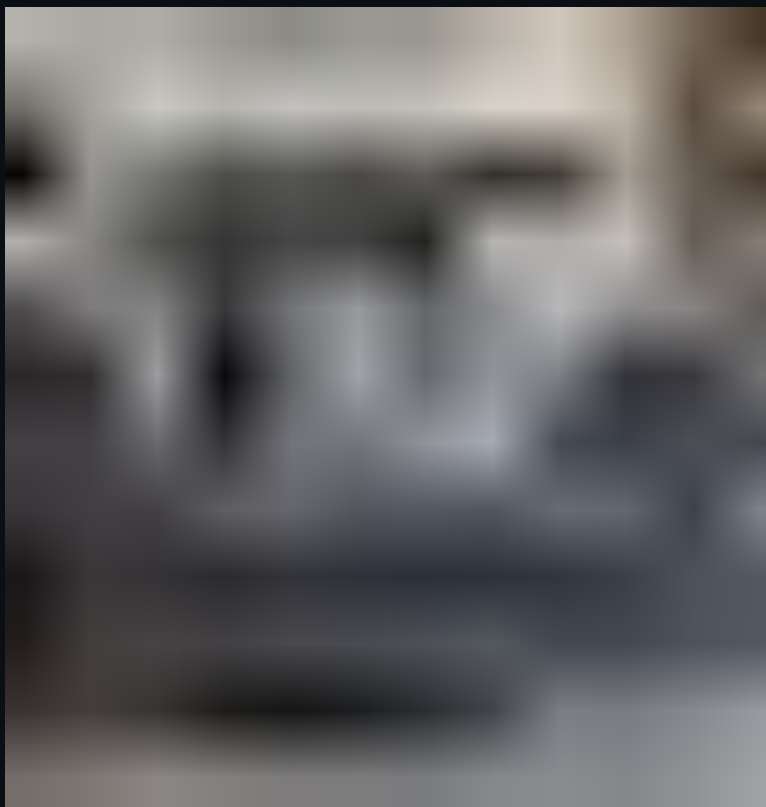
LAYER 02

Output

// what the model generates
1% of the token volume

- Verbosity & filler
- Repetitive scaffolding
- Polite preambles
- Over-explanation
- Markdown overhead

95% reduction



What do you see?

Original size



And now?

Original size



Compression is lossy. The question is whether it is lossy enough to *cost you a turn*.

≡ edgee

The dial nobody **calibrates**.



`saving = ( tokens removed ) - ( cost of the extra turns it caused )`

`and an extra turn re-bills the ENTIRE accumulated context`

What the community **built**.

// TOOL & CLI OUTPUT

RTK

CLI proxy + hook. Rewrites shell output before context. 79k stars.

Context Mode

MCP. Runs output in a sandbox, returns summaries.

// CODEBASE NAVIGATION

Graphify

Dependency graph + LLM pass. Understands synonyms.

CCE

Index the codebase, retrieve chunks rather than files.

LSP

Not a plugin. The language servers you already have.

// CONTEXT / PROXY

Headroom

HTTP proxy. Compresses everything the agent reads.

lean-ctx

Context trimming CLI.

// OUTPUT SHAPE

Caveman

Skill. Kills filler. Code and commands stay byte-exact.

Ponytail

Skill. A YAGNI ladder before writing any code.

token-efficient

One CLAUDE.md drop-in.

What the four most popular **promise**.

// TOOL	// ADVERTISED	
<u>RTK</u>	- 60 to 90 %	on common dev commands
<u>Caveman</u>	- 65 %	of tokens
<u>Ponytail</u>	- 20 %	cost
<u>Headroom</u>	- 60 to 95 %	fewer tokens

All figures taken from each project's repositories September 2026.

What third parties **measured**.

// TOOL	// ADVERTISED	// MEASURED, PAIRED A/B
<u>RTK</u>	– 60 to 90 %	+ 7.6 % MORE expensive (JetBrains, p=0.004) + 17 % per task on DeepSeek (Quesma) no effect (Marmelab)
<u>Caveman</u>	– 65 %	– 8.5 % output tokens, skill forced on (JetBrains) + 3 to + 104 % on a real agent loop (Marmelab)
<u>Ponytail</u>	– 20 %	– 10.3 % cost (JetBrains, p=0.004) the only clear saving in the series
<u>Headroom</u>	– 60 to 95 %	+ 52.8 % MORE expensive, last of 12 (THOL) + 32 % in token mode (Marmelab)

<https://marmelab.com/blog/2026/08/27/which-agent-based-plugin-should-you-use-in-2026.html>

<https://quesma.com/blog/does-rtk-make-ai-coding-cheaper/>

<https://blog.jetbrains.com/ai/2026/07/rtk-claude-code-token-savings/>

<https://pi-infected.github.io/token-harness-optimizer-leaderboard/>

<https://blog.jetbrains.com/ai/2026/07/speak-to-ai-agents-like-cavemen-to-save-tokens/>

<https://blog.jetbrains.com/ai/2026/07/ponytail-skill-claude-tested/>

Why this gap?

- | | | |
|----|-----------------------|--|
| 01 | Optimistic arithmetic | Some tools count the percentage of tokens they remove within the compressible area, not the entire token surface. |
| 02 | Wrong token counting | The counter is not counting tokens. Most of the time, these tools consider 4 characters as a token. No model tokenizer is ever called. |
| 03 | Standing overhead | Injected instructions are re-billed on every single turn. Caveman saved output and paid for it in context. |
| 04 | Cache arithmetic | 94% of input is cache reads, billed at 0.1x. You are compressing the cheap tokens. And rewriting the history to compress it turns those reads into writes at 1.25x. A 12x swing. |
| 05 | Extra turns | Lossy compression makes the model re-run the command. |

Edgee compression layers, a scientific approach

Tool Result Trimming

Fork of RTK and adapted for our gateway. Filters tool_result messages before they reach the model

Tool Surface Reduction

Coding agents connect multiple MCP servers, each exposing its own set of tools. The agent sends the full tool list to the model on every request, even when only one or two MCP servers are relevant. This bloats context and drives up cost.

Output Brevity

Reduces verbosity in model responses without losing technical content. Same answer, fewer tokens. Single flag, no level parameter.



Built, tested and evaluated with [SWE-Bench](#) by

Khaled Maâmra PhD

Research Scientist

Demo. Compression.

```
$ curl -fsSL https://edgee.ai/install.sh | bash
```

```
$ edgee launch claude
```

So where is the **money**?

// COMPRESSION, HONESTLY MEASURED

15-20%

Token compression only saves you a fraction, but you should still use it.

So where is the **money**?

// COMPRESSION, HONESTLY MEASURED

15-20%

Token compression only saves you a fraction, but you should still use it.

// ROUTING, FRONTIER vs OPEN WEIGHT

~**10X**

The choice of model to use for each task has an impact of different magnitude

Routing

A cheaper model changes the economics.

100 accepted tasks in both cases

Baseline	\$100	100 tasks at \$1	All on the initial model
Routed work	\$46	$60 \times \$0.10 + 40 \times \1	Same 100 accepted tasks
Recovery cost	+\$10	Retries and escalation	Included in total spend
Final result	\$56	44% lower cost	\$0.56 per accepted task

Illustrative task costs, not model prices or customer results. Excludes subscriptions and gateway fees.

Open-weight models, the new frontier.

OPEN WEIGHTS		Terminal-Bench 2.1 Reported score	Input	Output
	GLM-5.3	88.2%	\$1.40	\$4.40
	DeepSeek V4 Pro	87.9%	\$1.32	\$3.96
	Kimi K3	88.3%	\$3.00	\$15.00
CLOSED WEIGHTS				
	Claude Opus 4.8	85.0%	\$5.00	\$25.00
	Claude Fable 5	88.0%	\$10.00	\$50.00
	GPT-5.6 Sol	88.8%	\$4.00	\$20.00

Three **routing** decisions.

- | | | |
|----|---------------------------|--|
| 01 | Fallback | On a provider error or limit, try an eligible fallback before a response begins. |
| 02 | Rerouting | Replace the requested model on every request for a chosen scope. |
| 03 | Routing strategies | Change the serving model as a developer or squad consumes its budget. |

Demo. Fallback & Reroute

A controlled failure. A verified fallback. The task continues.

```
$ edgee launch claude
```

Watch the serving model, the event log and the final tests.

A budget becomes a **routing policy**.

Example: \$2000 monthly budget, shared by one squad

0-60%	Requested model
60-85%	Lower-cost model
85-100%	Evaluated final model

// AT 100%

Stay on last model:
spend continues.

Or block requests:
work stops.

Illustrative thresholds. Model compatibility and quality need evaluation.

Demo. Routing strategies

The next request follows the active budget stage.

```
$ edgee launch claude
```

Edgee console: strategy, squad and session

What to do Monday morning.

1. Measure

One squad, one repo, representative tasks

2. Route

Evaluate a fallback and a budget policy

3. Compress

One technique at a time, against the baseline

4. Expand

Keep lower cost, accepted quality and latency

Questions?

sacha@edgee.ai

edgee.ai/u/sacha